

An Unbounded Decimal Floating-Point Type

Doc No: P2159R0

Date: 2020-04-23

Author: Bill Seymour

Reply to: stdbill.h@pobox.com

Audience: LEWG-I, SG6 (Numerics)

Is there any interest in an unbounded decimal floating-point type for the numbers TS proposed in P1889?

(The author intends to implement an open-source version for his own use in any event. Should he write the standardese?)

This paper uses from [P1889R1](#) `enum class rounding` (§3.3), `integer` (§9.3), and `decimal128` (IEEE 754 decimal floating-point, future §8.5?).

The “scale” is the number of digits to the right of the decimal point. The rounding mode defaults to `tie_to_even` but can be changed at any time.

Note the arguments in [P2010](#) for `std::format`, `std::to_chars` and `std::from_chars` which we might want to add. (P2010’s arguments against the I/O operators probably don’t apply since those operators could be written to use the `numpunct` facet from the stream’s imbued locale, and they could be optional in freestanding environments.)

The basic idea:

(`std::` qualifiers omitted for simplicity)

```
class decimal
{
public:
    //
    // As a practical matter, we'll need to limit the number of fractional digits
    // to something finite.
    //
    static constexpr int max_scale = implementation-defined;

    //
    // Constructors, destructor:
    //
    constexpr decimal() noexcept;

    decimal(long double); // NB: implicit
    decimal(decimal128);

    decimal(const integer& unscaled_value, int scale);
    decimal(integer&& unscaled_value, int scale) noexcept;

    // as if in "C" locale:
    template<class Ch, class Tr, class A>
        explicit decimal(const basic_string<Ch,Tr,A>&, int radix = 10);

    ~decimal() noexcept;

    //
```

```

// Copyable, noexcept-moveable, noexcept-swappable:
//
decimal(const decimal&);
decimal(decimal&&) noexcept;

decimal& operator=(const decimal&);
decimal& operator=(decimal&&) noexcept;

void swap(decimal&) noexcept;

//
// Capacity in units of decimal digits:
//
size_t size() const noexcept;
size_t capacity() const noexcept;
void reserve(size_t digits);
void shrink_to_fit();

//
// Conversions:
//
explicit operator bool() const noexcept;
explicit operator integer() const;
explicit operator long double() const noexcept;
explicit operator decimal128() const noexcept;

// as if in "C" locale:
string to_string(int scale = -1, // < 0 means exact
                int radix = 10) const;

//
// Observers:
//
const integer& unscaled_value() const noexcept;
int scale() const noexcept;
rounding rounding_mode() const noexcept;

bool is_zero() const noexcept;
bool is_neg() const noexcept;

decimal integer_part() const;    // rounding::all_to_zero
decimal fractional_part() const; // *this - integer_part()
decimal modf(decimal*) const;    // after modf(double,double*)

//
// Modifiers:
//
void rescale(int new_scale) noexcept; // shifts decimal point in O(1) time
void reset_rounding(rounding = rounding::tie_to_even) noexcept;

//
// Comparisons:
//
int compare(const decimal&) const noexcept;

//
// Member operators:
//
decimal& operator++();
decimal& operator--();

decimal operator++(int);
decimal operator--(int);

decimal& operator+=(const decimal&);

```

```

    decimal& operator-=(const decimal&);
    decimal& operator*=(const decimal&);
    decimal& operator/=(const decimal&);

    //
    // Other arithmetic operations:
    //
    decimal& abs() noexcept;
    decimal& negate() noexcept;
    decimal& sqrt();
    decimal& pow(const decimal& exponent);
    decimal& remainder(const decimal& divisor);
};
void swap(decimal&, decimal&) noexcept;

//
// Non-member operators:
//
decimal operator+(const decimal&);
decimal operator+(decimal&&) noexcept;

decimal operator-(const decimal&);
decimal operator-(decimal&&) noexcept;

decimal operator+(const decimal&, const decimal&);
decimal operator-(const decimal&, const decimal&);
decimal operator*(const decimal&, const decimal&);
decimal operator/(const decimal&, const decimal&);

//
// Other non-member arithmetic operations:
//
decimal abs(const decimal&);
decimal abs(decimal&&) noexcept;

decimal sqrt(const decimal&);

decimal pow(const decimal& base, const decimal& exponent);

decimal remainder(const decimal& dividend, const decimal& divisor);

//
// I/O:
//
template<class Ch, class Tr>
    basic_ostream<Ch,Tr>& operator<<(basic_ostream<Ch,Tr>&, const decimal&);

template<class Ch, class Tr>
    basic_istream<Ch,Tr>& operator>>(basic_istream<Ch,Tr>&, decimal&);

```

All corrections and suggestions will be welcome. All flames will be amusing.

Mail to: stdbill.h@pobox.com